🛒 CodeMart Backend API (CodeMart.Server)

A robust, scalable .NET 8.0 ASP.NET Core Web API serving as the backend for the CodeMart platform—a marketplace for buying and selling software projects.

📖 Overview

CodeMart.Server handles all business logic, data persistence, and secure communications for the CodeMart platform. It exposes a RESTful API for user authentication, project management, shopping cart operations, checkout processing, and user reviews.

✨ Key Features

- **User Authentication & Authorization**: Secure JWT-based authentication with role-based access control (RBAC). Passwords are encrypted using BCrypt.
- **Project Marketplace**: Endpoints for browsing, filtering, uploading, and managing software projects/products.
- **Order & Cart Management**: Full shopping cart functionality with seamless checkout flows.
- **Payment Integration**: Secure payment processing utilizing **Stripe**.
- **Reviews & Ratings**: Buyers can leave ratings and reviews for purchased projects.
- **Transactions & History**: Transparent tracking of transaction statuses and user purchase histories.

🛠️ Tech Stack

- **Framework**: .NET 8.0 (ASP.NET Core Web API)
- **Language**: C# 12
- **ORM**: Entity Framework Core 9
- **Database**: PostgreSQL (hosted on Supabase)
- **Authentication**: JWT (JSON Web Tokens)
- **Payment Gateway**: Stripe.net
- **Documentation**: Swagger / OpenAPI
- **Containerization**: Docker & Docker Compose

🧬 Architecture & Structure

The repository follows a clean, service-oriented architecture:

- `Controllers/`: API endpoints handling HTTP requests and routing.
- `Services/`: Core business logic and service layer.
- `Models/`: Domain entities representing database tables.
- `DTOs/`: Data Transfer Objects for API request/response payloads.
- `Data/`: Entity Framework Core DbContext and configurations.
- `Interfaces/`: Abstractions for services promoting dependency injection.
- `Utils/`: Helper classes and extension methods.

🚀 Quick Start (Development)

Prerequisites

- **.NET 8.0 SDK**
- **Docker Desktop** (optional, for containerized DB/App setup)
- **Git**
- **Supabase Account** (for database)

Step 1: Clone the Repository

```
git clone <your-repository-url>
cd CodeMart/CodeMart.Server
```

Step 2: Environment Variables

Create a `.env` file in the root of the `CodeMart.Server` directory based on the `.env.example`:

```
# On Windows
Copy-Item .env.example .env
# On Linux/macOS
cp .env.example .env
```

Required `.env` Variables:

- `DB_CONNECTION_STRING`: Your Supabase Postgres connection string.
- `JWT_KEY`: A 32+ character secret string for signing JWT tokens.
- `SUPABASE_PROJECT_URL`: Your Supabase API URL.
- `SUPABASE_API_KEY`: Your Supabase API key.
- `STRIPE_SECRET_KEY`: Your Stripe API secret key.

Step 3: Run the API Locally

Option A: Using the .NET CLI

```
dotnet restore
dotnet run
```

Option B: Using Docker Compose

```
docker-compose up -d
docker-compose logs -f api
```

The API will be available at `http://localhost:8080`.

Step 4: Run Database Migrations

Apply EF Core migrations to sync your Supabase PostgreSQL database:

```
# Using .NET CLI
dotnet ef database update

# If running via Docker (Windows)
.\docker-migrate.ps1
```

API Documentation (Swagger)

When running in Development mode, the API provides interactive Swagger documentation.

- **Swagger UI**: `http://localhost:8080/swagger`

You can use the Swagger UI to view all available endpoints, required payload schemas, and to test requests directly from your browser. Ensure you authenticate by pasting a generated JWT token into the **Authorize** lock icon.

Production Deployment

The backend is fully containerized and production-ready.

1. Ensure your `.env` contains production secrets (e.g., strong `JWT_KEY`, live Stripe keys).
2. Start the production stack:

   ```
   docker-compose -f docker-compose.yml up -d
   ```

3. Run migrations inside the container:

   ```
   docker-compose exec api dotnet ef database update
   ```

(See DOCKER.md and ENV_SETUP.md for more advanced deployment guides.)

Database Management

We leverage **Supabase** for fully managed PostgreSQL hosting.

- **Connection String**: `Host=db.[project-ref].supabase.co;Port=5432;Database=postgres;Username=postgres;Password=[password];SSL Mode=Require`
- You can manage tables and rows directly from the Supabase Studio dashboard.

Contributing

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

License

This project is licensed under the MIT License - see the LICENSE file for details.